

*Fundamentals of*  
**Database  
Systems**

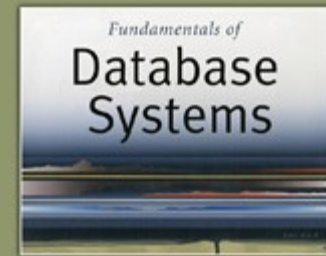


5<sup>th</sup> Edition

Elmasri / Navathe

# Chapter 10

## Functional Dependencies and Normalization for Relational Databases



5th Edition

Elmasri / Navathe

# Chapter Outline

- 1 Informal Design Guidelines for Relational Databases
  - 1.1 Semantics of the Relation Attributes
  - 1.2 Redundant Information in Tuples and Update Anomalies
  - 1.3 Null Values in Tuples
  - 1.4 Spurious Tuples
- 2 Functional Dependencies (FDs)
  - 2.1 Definition of FD
  - 2.2 Inference Rules for FDs
  - 2.3 Equivalence of Sets of FDs
  - 2.4 Minimal Sets of FDs

# Chapter Outline

- 3 Normal Forms Based on Primary Keys
  - 3.1 Normalization of Relations
  - 3.2 Practical Use of Normal Forms
  - 3.3 Definitions of Keys and Attributes Participating in Keys
  - 3.4 First Normal Form
  - 3.5 Second Normal Form
  - 3.6 Third Normal Form
- 4 General Normal Form Definitions (For Multiple Keys)
- 5 BCNF (Boyce-Codd Normal Form)

# 1 Informal Design Guidelines for Relational Databases (1)

- What is relational database design?
  - The grouping of attributes to form "good" relation schemas
- Two levels of relation schemas
  - The logical "user view" level
  - The storage "base relation" level
- Design is concerned mainly with base relations
- What are the criteria for "good" base relations?

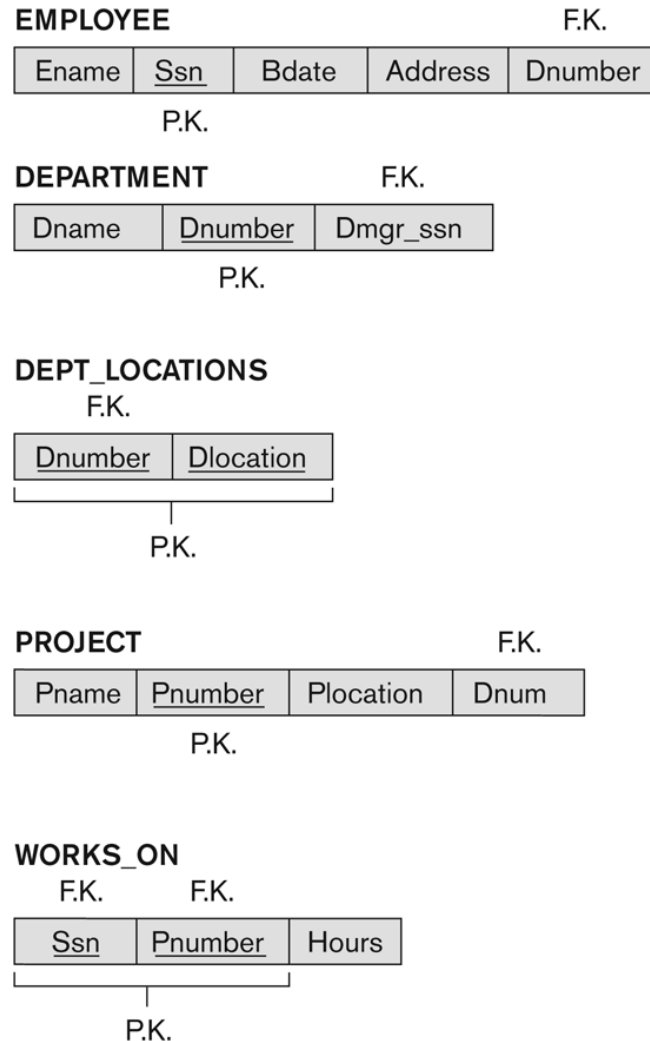
# Informal Design Guidelines for Relational Databases (2)

- We first discuss informal guidelines for good relational design
- Then we discuss formal concepts of functional dependencies and normal forms
  - - 1NF (First Normal Form)
  - - 2NF (Second Normal Form)
  - - 3NF (Third Normal Form)
  - - BCNF (Boyce-Codd Normal Form)
- Additional types of dependencies, further normal forms, relational design algorithms by synthesis are discussed in Chapter 11

# 1.1 Semantics of the Relation Attributes

- GUIDELINE 1: Informally, each tuple in a relation should represent one entity or relationship instance. (Applies to individual relations and their attributes).
  - Attributes of different entities (EMPLOYEEs, DEPARTMENTs, PROJECTs) should not be mixed in the same relation
  - Only foreign keys should be used to refer to other entities
  - Entity and relationship attributes should be kept apart as much as possible.
- Bottom Line: *Design a schema that can be explained easily relation by relation. The semantics of attributes should be easy to interpret.*

# Figure 10.1 A simplified COMPANY relational database schema



**Figure 10.1**  
A simplified COMPANY  
relational database  
schema.



# 1.2 Redundant Information in Tuples and Update Anomalies

- Information is stored redundantly
  - Wastes storage
  - Causes problems with update anomalies
    - Insertion anomalies
    - Deletion anomalies
    - Modification anomalies

# EXAMPLE OF AN UPDATE ANOMALY

- Consider the relation:
  - EMP\_PROJ(Emp#, Proj#, Ename, Pname, No\_hours)
- Update Anomaly:
  - Changing the name of project number P1 from “Billing” to “Customer-Accounting” may cause this update to be made for all 100 employees working on project P1.

# EXAMPLE OF AN INSERT ANOMALY

- Consider the relation:
  - EMP\_PROJ(Emp#, Proj#, Ename, Pname, No\_hours)
- Insert Anomaly:
  - Cannot insert a project unless an employee is assigned to it.
- Conversely
  - Cannot insert an employee unless an he/she is assigned to a project.

# EXAMPLE OF AN DELETE ANOMALY

- Consider the relation:
  - EMP\_PROJ(Emp#, Proj#, Ename, Pname, No\_hours)
- Delete Anomaly:
  - When a project is deleted, it will result in deleting all the employees who work on that project.
  - Alternately, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project.

# Figure 10.3 Two relation schemas suffering from update anomalies

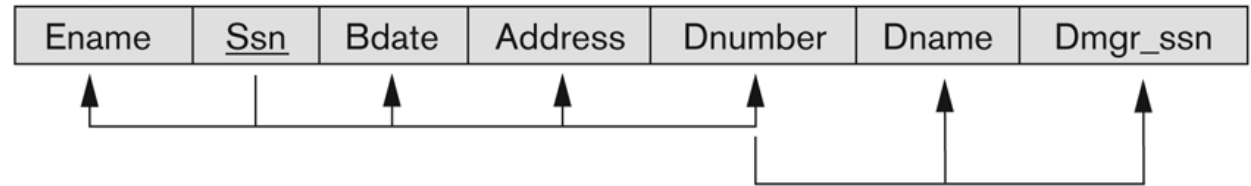
**Figure 10.3**

Two relation schemas suffering from update anomalies.

(a) EMP\_DEPT and  
(b) EMP\_PROJ.

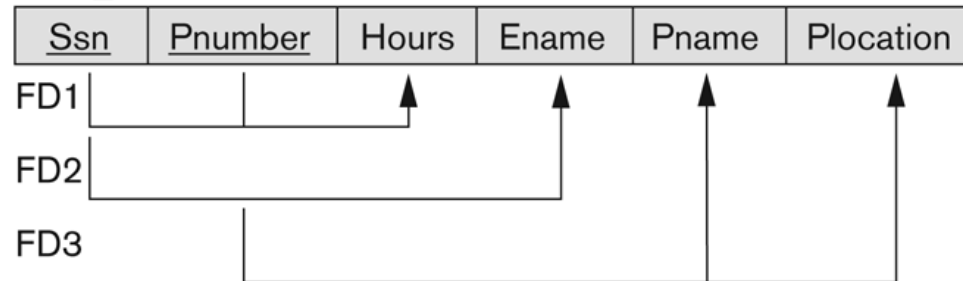
(a)

**EMP\_DEPT**



(b)

**EMP\_PROJ**



# Figure 10.4 Example States for EMP\_DEPT and EMP\_PROJ

**Figure 10.4**

Example states for EMP\_DEPT and EMP\_PROJ resulting from applying NATURAL JOIN to the relations in Figure 10.2. These may be stored as base relations for performance reasons.

| EMP_DEPT             |           |            |                          |         |                |           | Redundancy |  |
|----------------------|-----------|------------|--------------------------|---------|----------------|-----------|------------|--|
| Ename                | Ssn       | Bdate      | Address                  | Dnumber | Dname          | Dmgr_ssn  |            |  |
| Smith, John B.       | 123456789 | 1965-01-09 | 731 Fondren, Houston, TX | 5       | Research       | 333445555 |            |  |
| Wong, Franklin T.    | 333445555 | 1955-12-08 | 638 Voss, Houston, TX    | 5       | Research       | 333445555 |            |  |
| Zelaya, Alicia J.    | 999887777 | 1968-07-19 | 3321 Castle, Spring, TX  | 4       | Administration | 987654321 |            |  |
| Wallace, Jennifer S. | 987654321 | 1941-06-20 | 291 Berry, Bellaire, TX  | 4       | Administration | 987654321 |            |  |
| Narayan, Ramesh K.   | 666884444 | 1962-09-15 | 975 FireOak, Humble, TX  | 5       | Research       | 333445555 |            |  |
| English, Joyce A.    | 453453453 | 1972-07-31 | 5631 Rice, Houston, TX   | 5       | Research       | 333445555 |            |  |
| Jabbar, Ahmad V.     | 987987987 | 1969-03-29 | 980 Dallas, Houston, TX  | 4       | Administration | 987654321 |            |  |
| Borg, James E.       | 888665555 | 1937-11-10 | 450 Stone, Houston, TX   | 1       | Headquarters   | 888665555 |            |  |

| EMP_PROJ  |         |       |                      |                 |           | Redundancy |  | Redundancy |  |
|-----------|---------|-------|----------------------|-----------------|-----------|------------|--|------------|--|
| Ssn       | Pnumber | Hours | Ename                | Pname           | Plocation |            |  |            |  |
| 123456789 | 1       | 32.5  | Smith, John B.       | ProductX        | Bellaire  |            |  |            |  |
| 123456789 | 2       | 7.5   | Smith, John B.       | ProductY        | Sugarland |            |  |            |  |
| 666884444 | 3       | 40.0  | Narayan, Ramesh K.   | ProductZ        | Houston   |            |  |            |  |
| 453453453 | 1       | 20.0  | English, Joyce A.    | ProductX        | Bellaire  |            |  |            |  |
| 453453453 | 2       | 20.0  | English, Joyce A.    | ProductY        | Sugarland |            |  |            |  |
| 333445555 | 2       | 10.0  | Wong, Franklin T.    | ProductY        | Sugarland |            |  |            |  |
| 333445555 | 3       | 10.0  | Wong, Franklin T.    | ProductZ        | Houston   |            |  |            |  |
| 333445555 | 10      | 10.0  | Wong, Franklin T.    | Computerization | Stafford  |            |  |            |  |
| 333445555 | 20      | 10.0  | Wong, Franklin T.    | Reorganization  | Houston   |            |  |            |  |
| 999887777 | 30      | 30.0  | Zelaya, Alicia J.    | Newbenefits     | Stafford  |            |  |            |  |
| 999887777 | 10      | 10.0  | Zelaya, Alicia J.    | Computerization | Stafford  |            |  |            |  |
| 987987987 | 10      | 35.0  | Jabbar, Ahmad V.     | Computerization | Stafford  |            |  |            |  |
| 987987987 | 30      | 5.0   | Jabbar, Ahmad V.     | Newbenefits     | Stafford  |            |  |            |  |
| 987654321 | 30      | 20.0  | Wallace, Jennifer S. | Newbenefits     | Stafford  |            |  |            |  |
| 987654321 | 20      | 15.0  | Wallace, Jennifer S. | Reorganization  | Houston   |            |  |            |  |
| 888665555 | 20      | Null  | Borg, James E.       | Reorganization  | Houston   |            |  |            |  |

# Guideline to Redundant Information in Tuples and Update Anomalies

- **GUIDELINE 2:**
  - Design a schema that does not suffer from the insertion, deletion and update anomalies.
  - If there are any anomalies present, then note them so that applications can be made to take them into account.

# 1.3 Null Values in Tuples

- **GUIDELINE 3:**
  - Relations should be designed such that their tuples will have as few NULL values as possible
  - Attributes that are NULL frequently could be placed in separate relations (with the primary key)
- **Reasons for nulls:**
  - Attribute not applicable or invalid
  - Attribute value unknown (may exist)
  - Value known to exist, but unavailable



# 1.4 Spurious Tuples

- Bad designs for a relational database may result in erroneous results for certain JOIN operations
- The "lossless join" property is used to guarantee meaningful results for join operations
- **GUIDELINE 4:**
  - The relations should be designed to satisfy the lossless join condition.
  - No spurious tuples should be generated by doing a natural-join of any relations.

# Spurious Tuples (2)

- There are two important properties of decompositions:
  - a) Non-additive or losslessness of the corresponding join
  - b) Preservation of the functional dependencies.
  
- Note that:
  - Property (a) is extremely important and *cannot* be sacrificed.
  - Property (b) is less stringent and may be sacrificed. (See Chapter 11).

## 2.1 Functional Dependencies (1)

- Functional dependencies (FDs)
  - Are used to specify *formal measures* of the "goodness" of relational designs
  - And keys are used to define **normal forms** for relations
  - Are **constraints** that are derived from the *meaning* and *interrelationships* of the data attributes
- A set of attributes  $X$  *functionally determines* a set of attributes  $Y$  if the value of  $X$  determines a unique value for  $Y$

# Functional Dependencies (2)

- $X \rightarrow Y$  holds if whenever two tuples have the same value for  $X$ , they *must have* the same value for  $Y$ 
  - For any two tuples  $t1$  and  $t2$  in any relation instance  $r(R)$ : If  $t1[X]=t2[X]$ , *then*  $t1[Y]=t2[Y]$
- $X \rightarrow Y$  in  $R$  specifies a *constraint* on all relation instances  $r(R)$
- Written as  $X \rightarrow Y$ ; can be displayed graphically on a relation schema as in Figures. ( denoted by the arrow: ).
- FDs are derived from the real-world constraints on the attributes

# Examples of FD constraints (1)

- Social security number determines employee name
  - $SSN \rightarrow ENAME$
- Project number determines project name and location
  - $PNUMBER \rightarrow \{PNAME, PLOCATION\}$
- Employee ssn and project number determines the hours per week that the employee works on the project
  - $\{SSN, PNUMBER\} \rightarrow HOURS$

# Examples of FD constraints (2)

- An FD is a property of the attributes in the schema  $R$
- The constraint must hold on *every* relation instance  $r(R)$
- If  $K$  is a key of  $R$ , then  $K$  functionally determines all attributes in  $R$ 
  - (since we never have two distinct tuples with  $t1[K]=t2[K]$ )

## 2.2 Inference Rules for FDs (1)

- Given a set of FDs  $F$ , we can **infer** additional FDs that hold whenever the FDs in  $F$  hold
- Armstrong's inference rules:
  - IR1. (**Reflexive**) If  $Y$  *subset-of*  $X$ , then  $X \rightarrow Y$
  - IR2. (**Augmentation**) If  $X \rightarrow Y$ , then  $XZ \rightarrow YZ$ 
    - (Notation:  $XZ$  stands for  $X \cup Z$ )
  - IR3. (**Transitive**) If  $X \rightarrow Y$  and  $Y \rightarrow Z$ , then  $X \rightarrow Z$
- IR1, IR2, IR3 form a **sound** and **complete** set of inference rules
  - These are rules hold and all other rules that hold can be deduced from these

# Inference Rules for FDs (2)

- Some additional inference rules that are useful:
  - **Decomposition:** If  $X \rightarrow YZ$ , then  $X \rightarrow Y$  and  $X \rightarrow Z$
  - **Union:** If  $X \rightarrow Y$  and  $X \rightarrow Z$ , then  $X \rightarrow YZ$
  - **Pseudotransitivity:** If  $X \rightarrow Y$  and  $WY \rightarrow Z$ , then  $WX \rightarrow Z$
- The last three inference rules, as well as any other inference rules, can be deduced from IR1, IR2, and IR3 (completeness property)



# Inference Rules for FDs (3)

- **Closure** of a set  $F$  of FDs is the set  $F^+$  of all FDs that can be inferred from  $F$
- **Closure** of a set of attributes  $X$  with respect to  $F$  is the set  $X^+$  of all attributes that are functionally determined by  $X$
- $X^+$  can be calculated by repeatedly applying IR1, IR2, IR3 using the FDs in  $F$

**Ex. 1:** Consider a relation schema having attributes A, B, C. Given the FD  $\{A \rightarrow B, B \rightarrow C\}$ . Determine the possible candidate keys.

$$\{A\}^+ = \{A, B, C\}$$

$$\{B\}^+ = \{B, C\}$$

$$\{C\}^+ = \{C\}$$

$$\{AB\}^+ = \{A, B, C\}$$

*Bi extraneous*

~~A~~, AB, AC, ABC

$A \rightarrow B$   
 $A \rightarrow C ?$

$B \rightarrow A ? \times$

$C \rightarrow ? \times$

candidate key is a minimal super key  
 super key is an attribute(s) whose closure covers all attributes of the relation schema

**Ex. 2:** Consider a relation schema having attributes A, B, C, D, E. Given the FD  $\{A \rightarrow BC, C \rightarrow B, D \rightarrow E, E \rightarrow D\}$ . Determine the possible candidate keys.

$$\{A\}^+ = \{A, B, C\}$$

$$\{B\}^+ = \{B\}$$

$$\{C\}^+ = \{C, B\}$$

$$\{D\}^+ = \{D, E\}$$

$$\{E\}^+ = \{E, D\}$$

$$B \rightarrow ?$$

$$\{AB\}^+ = \{A, B, C\}$$

$$\cancel{D, E} \text{ or } E$$

$$\{AD\}^+ = \{A, D, B, C, E\}$$

AD is a possible super key

$$AD, AE$$

$$\underline{AD} \text{ or } \underline{AE}$$

are the possible candidate keys

$$\{AE\}^+ = \{A, E, B, C, D\}$$

$$\{ADE\}^+ = \{A, \dots, E\}$$

**Ex. 3:** Consider a relation schema having attributes A, B, C, D, E.  
 Given the FD  $\{A \rightarrow B, A \rightarrow C, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$ . Which of the following functional dependencies is not implied by the above set?

(a)  $CD \rightarrow AC$  (b)  $BD \rightarrow CD$  (c)  $BC \rightarrow CD$  (d)  $AC \rightarrow BC$

To check whether a rule  $A \rightarrow B$  can be implied/derived from given set of FD,
   
 (1) find  $\{A\}^+$  using FD  $\rightarrow$  If B is there in  $A^+$ 
  
 (2) If  $B$  is a subset of  $\{A\}^+$  then  $A \rightarrow B$  is implied / can be derived
   
 otherwise NO
   
 $\{CD\}^+ = \{C, D, E, A, B\}$ 
  
 $\{BD\}^+ = \{B, D\}$ 
  
 $\{BC\}^+ = \{B, C, D\}$ 
  
 $\{AC\}^+ = \{A, C, B, D\}$

## 2.3 Equivalence of Sets of FDs

- Two sets of FDs  $F$  and  $G$  are **equivalent** if:
  - Every FD in  $F$  can be inferred from  $G$ , and
  - Every FD in  $G$  can be inferred from  $F$
  - Hence,  $F$  and  $G$  are equivalent if  $F^+ = G^+$
- Definition (**Covers**):
  - $F$  **covers**  $G$  if every FD in  $G$  can be inferred from  $F$ 
    - (i.e., if  $G^+ \text{ subset-of } F^+$ )
- $F$  and  $G$  are equivalent if  $F$  covers  $G$  and  $G$  covers  $F$
- There is an algorithm for checking equivalence of sets of FDs

## 2.4 Minimal Sets of FDs (1)

- A set of FDs is **minimal** if it satisfies the following conditions:
  1. Every dependency in  $F$  has a single attribute for its RHS.
  2. We cannot remove any dependency from  $F$  and have a set of dependencies that is equivalent to  $F$ .
  3. We cannot replace any dependency  $X \rightarrow A$  in  $F$  with a dependency  $Y \rightarrow A$ , where  $Y$  proper-subset-of  $X$  ( $Y$  subset-of  $X$ ) and still have a set of dependencies that is equivalent to  $F$ .

# Minimal Sets of FDs (2)

- Every set of FDs has an equivalent minimal set
- There can be several equivalent minimal sets
- There is no simple algorithm for computing a minimal set of FDs that is equivalent to a set  $F$  of Fds

**Minimal cover:** The minimal cover is the set of FDs which are equivalent to the given FDs.

**Canonical cover:** In canonical cover, the LHS (Left Hand Side) must be unique

**Ex. 4:** Consider a relation schema having attributes A, B, C, D. Given the FD  $\{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$ . Find the Minimal Cover of FD.

First step: Make all FD in canonical form (single attribute in RHS)

key: No two FD should have same LHS

$A \rightarrow BC$   
 $B \rightarrow C$

$E =$

- $A \rightarrow B$
- $A \rightarrow C$
- $B \rightarrow C$
- $A \rightarrow B$
- $AB \rightarrow C$

$A, B \rightarrow C$

if  $E \subseteq F$  then we can say  $F$  is a minimal cover

Second step (Remove extraneous attribute from LHS)

$\checkmark A \rightarrow B$   
 $\checkmark B \rightarrow C$   
 $\times A \rightarrow C$   
 $\times A \rightarrow B$   
 $\times A \rightarrow C$

$\checkmark AB \rightarrow C$   
 $\checkmark A \rightarrow C$   
 $\checkmark B \rightarrow C$

Step 3 - Remove redundancy

$\{A \rightarrow B, B \rightarrow C\}$

Minimal cover

$F^+ = \{ \}$





# 3 Normal Forms Based on Primary Keys

- 3.1 Normalization of Relations
- 3.2 Practical Use of Normal Forms
- 3.3 Definitions of Keys and Attributes Participating in Keys
- 3.4 First Normal Form
- 3.5 Second Normal Form
- 3.6 Third Normal Form

# 3.1 Normalization of Relations (1)

- **Normalization:**

- The process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations

- **Normal form:**

- Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form

# Normalization of Relations (2)

- 2NF, 3NF, BCNF
  - based on keys and FDs of a relation schema
- 4NF
  - based on keys, multi-valued dependencies : MVDs; 5NF based on keys, join dependencies : JDs (Chapter 11)
- Additional properties may be needed to ensure a good relational design (lossless join, dependency preservation; Chapter 11)

## 3.2 Practical Use of Normal Forms

- **Normalization** is carried out in practice so that the resulting designs are of high quality and meet the desirable properties
- The practical utility of these normal forms becomes questionable when the constraints on which they are based are *hard to understand* or to *detect*
- The database designers *need not* normalize to the highest possible normal form
  - (usually up to 3NF, BCNF or 4NF)
- **Denormalization:**
  - The process of storing the join of higher normal form relations as a base relation—which is in a lower normal form

## 3.3 Definitions of Keys and Attributes Participating in Keys (1)

- A **superkey** of a relation schema  $R = \{A_1, A_2, \dots, A_n\}$  is a set of attributes  $S$  *subset-of*  $R$  with the property that no two tuples  $t_1$  and  $t_2$  in any legal relation state  $r$  of  $R$  will have  $t_1[S] = t_2[S]$
- A **key**  $K$  is a **superkey** with the *additional property* that removal of any attribute from  $K$  will cause  $K$  not to be a superkey any more.

# Definitions of Keys and Attributes Participating in Keys (2)

- If a relation schema has more than one key, each is called a **candidate** key.
  - One of the candidate keys is *arbitrarily* designated to be the **primary key**, and the others are called **secondary keys**.
- A **Prime attribute** must be a member of *some* candidate key
- A **Nonprime attribute** is not a prime attribute—that is, it is not a member of any candidate key.

## 3.2 First Normal Form

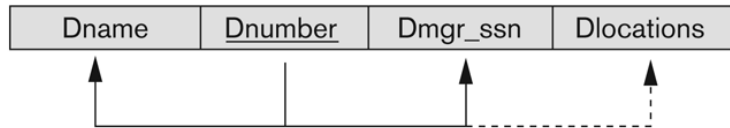
- Disallows
  - composite attributes
  - multivalued attributes
  - **nested relations**; attributes whose values for an *individual tuple* are non-atomic
- Considered to be part of the definition of relation



# Figure 10.8 Normalization into 1NF

(a)

**DEPARTMENT**



(b)

**DEPARTMENT**

| Dname          | <u>Dnumber</u> | Dmgr_ssn  | Dlocations                     |
|----------------|----------------|-----------|--------------------------------|
| Research       | 5              | 333445555 | {Bellaire, Sugarland, Houston} |
| Administration | 4              | 987654321 | {Stafford}                     |
| Headquarters   | 1              | 888665555 | {Houston}                      |

(c)

**DEPARTMENT**

| Dname          | <u>Dnumber</u> | Dmgr_ssn  | <u>Dlocation</u> |
|----------------|----------------|-----------|------------------|
| Research       | 5              | 333445555 | Bellaire         |
| Research       | 5              | 333445555 | Sugarland        |
| Research       | 5              | 333445555 | Houston          |
| Administration | 4              | 987654321 | Stafford         |
| Headquarters   | 1              | 888665555 | Houston          |

**Figure 10.8**

Normalization into 1NF.

(a) A relation schema that is not in 1NF. (b) Example state of relation DEPARTMENT. (c) 1NF version of the same relation with redundancy.

| DName    | DNumber | DMgr_Ssn | Phone No                                  |
|----------|---------|----------|-------------------------------------------|
| Research | 1       | 999      | { <del>9847680660</del> ,<br>7025515165 } |

⇓ NF

|          |   |     |
|----------|---|-----|
| Research | 1 | 999 |
| Research | 1 | 999 |

9847680660  
7025515165

# Figure 10.9 Normalization nested relations into 1NF

(a)

EMP\_PROJ

| Ssn | Ename | Projs   |       |
|-----|-------|---------|-------|
|     |       | Pnumber | Hours |

(b)

EMP\_PROJ

| Ssn       | Ename                | Pnumber | Hours |
|-----------|----------------------|---------|-------|
| 123456789 | Smith, John B.       | 1       | 32.5  |
|           |                      | 2       | 7.5   |
| 666884444 | Narayan, Ramesh K.   | 3       | 40.0  |
| 453453453 | English, Joyce A.    | 1       | 20.0  |
|           |                      | 2       | 20.0  |
| 333445555 | Wong, Franklin T.    | 2       | 10.0  |
|           |                      | 3       | 10.0  |
|           |                      | 10      | 10.0  |
|           |                      | 20      | 10.0  |
| 999887777 | Zelaya, AliciaJ.     | 30      | 30.0  |
|           |                      | 10      | 10.0  |
| 987987987 | Jabbar, Ahmad V.     | 10      | 35.0  |
|           |                      | 30      | 5.0   |
| 987654321 | Wallace, Jennifer S. | 30      | 20.0  |
|           |                      | 20      | 15.0  |
| 888665555 | Borg, James E.       | 20      | NULL  |

(c)

EMP\_PROJ1

| Ssn | Ename |
|-----|-------|
|-----|-------|

EMP\_PROJ2

| Ssn | Pnumber | Hours |
|-----|---------|-------|
|-----|---------|-------|

**Figure 10.9**

Normalizing nested relations into 1NF. (a) Schema of the EMP\_PROJ relation with a *nested relation* attribute PROJS. (b) Example extension of the EMP\_PROJ relation showing nested relations within each tuple. (c) Decomposition of EMP\_PROJ into relations EMP\_PROJ1 and EMP\_PROJ2 by propagating the primary key.

| EmpId | Emp Name | PNo. | Hours |
|-------|----------|------|-------|
|-------|----------|------|-------|

↓

Table 1

| EmpId | Emp Name |
|-------|----------|
|-------|----------|

Table 2

| EmpId | PNo. | Hours |
|-------|------|-------|
|-------|------|-------|

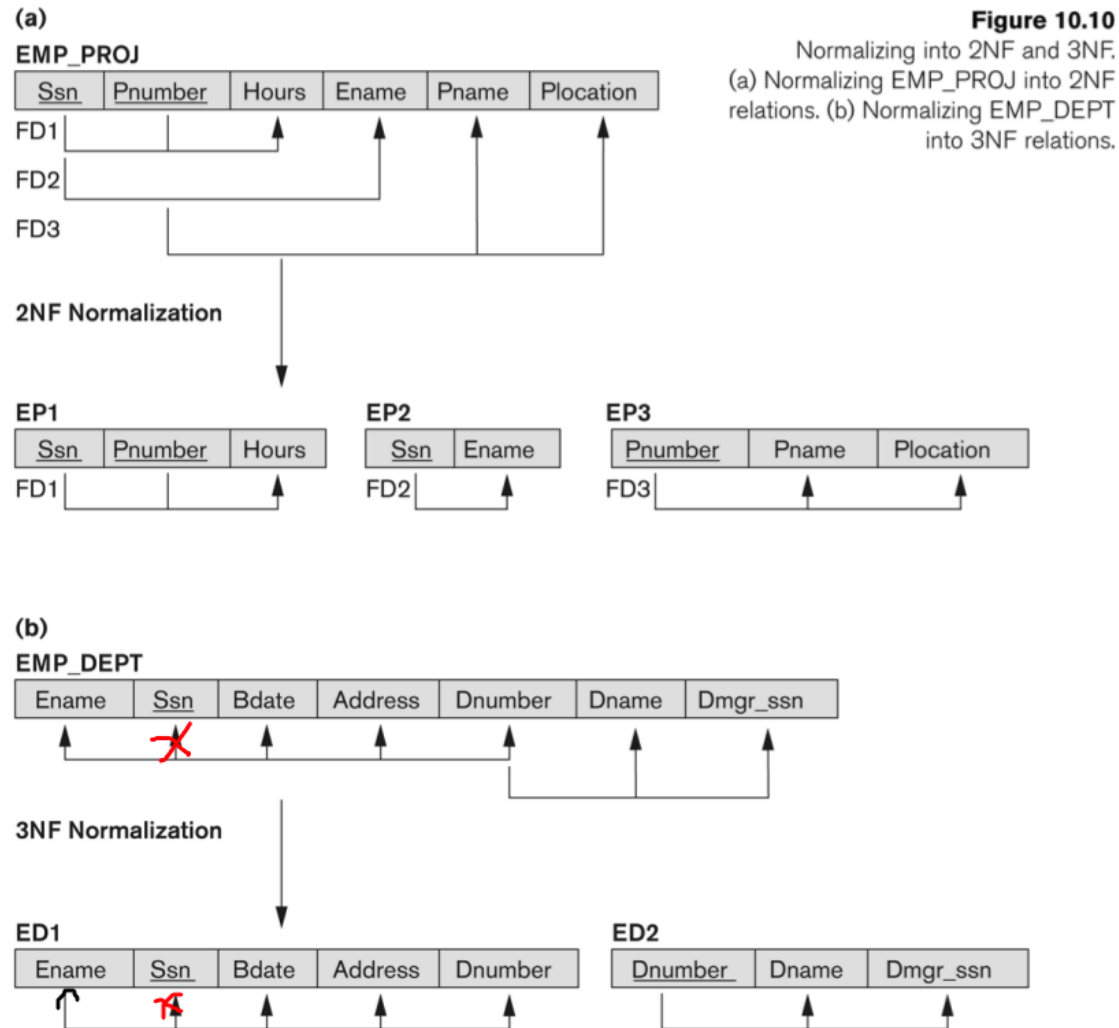
## 3.3 Second Normal Form (1)

- Uses the concepts of **FDs, primary key**
- Definitions
  - **Prime attribute:** An attribute that is member of the primary key K
  - **Full functional dependency:** a FD  $Y \rightarrow Z$  where removal of any attribute from Y means the FD does not hold any more
- Examples:
  - $\{SSN, PNUMBER\} \rightarrow HOURS$  is a full FD since neither  $SSN \rightarrow HOURS$  nor  $PNUMBER \rightarrow HOURS$  hold
  - $\{SSN, PNUMBER\} \rightarrow ENAME$  is not a full FD (it is called a partial dependency ) since  $SSN \rightarrow ENAME$  also holds

# Second Normal Form (2)

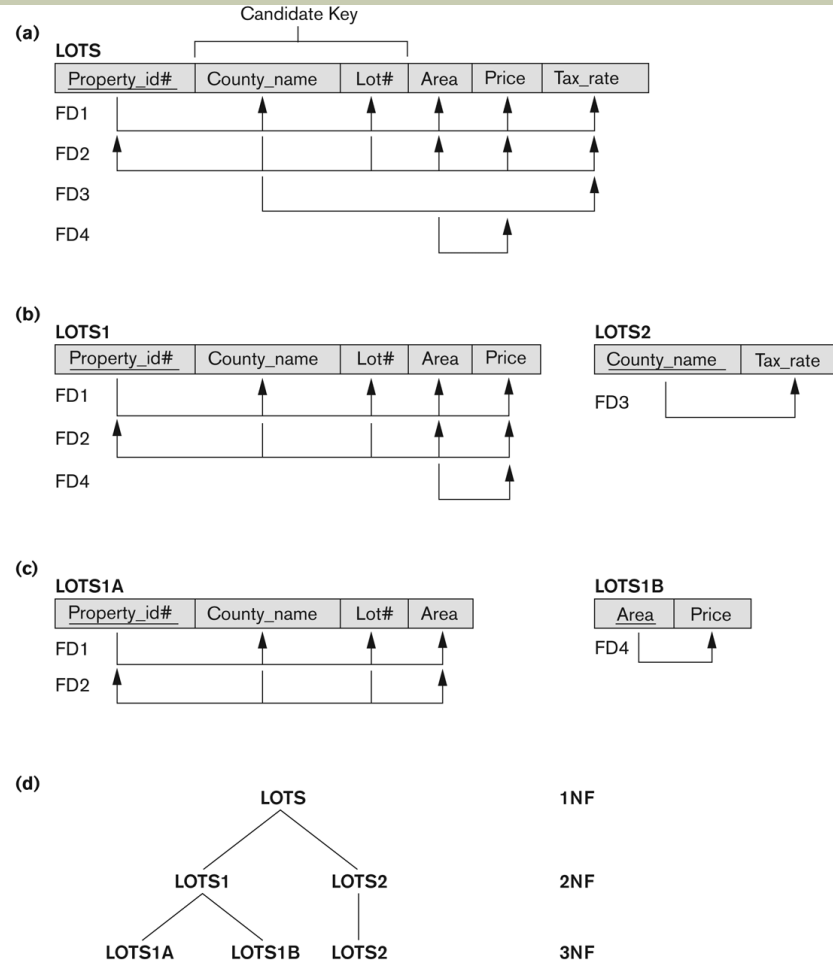
- A relation schema  $R$  is in **second normal form (2NF)** if every non-prime attribute  $A$  in  $R$  is fully functionally dependent on the primary key
- $R$  can be decomposed into 2NF relations via the process of 2NF normalization

# Figure 10.10 Normalizing into 2NF and 3NF



**Figure 10.10**  
 Normalizing into 2NF and 3NF.  
 (a) Normalizing EMP\_PROJ into 2NF relations.  
 (b) Normalizing EMP\_DEPT into 3NF relations.

# Figure 10.11 Normalization into 2NF and 3NF

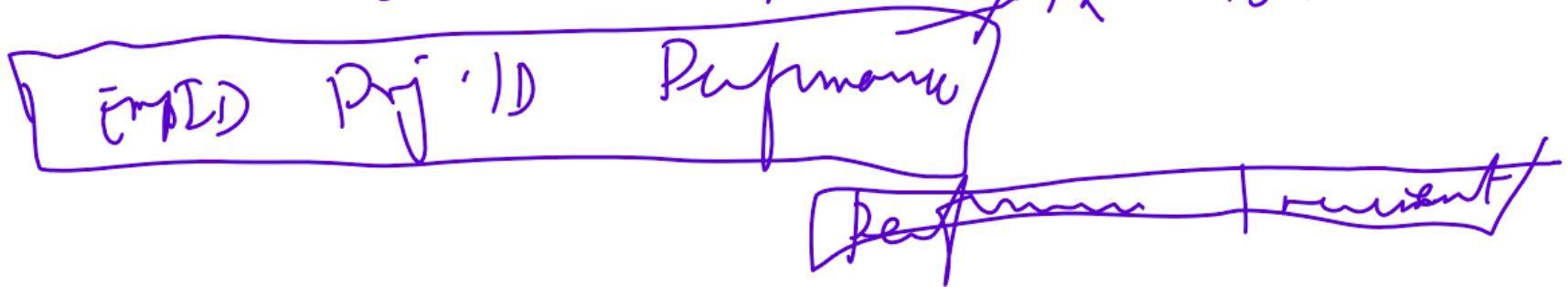


**Figure 10.11**

Normalization into 2NF and 3NF. (a) The LOTS relation with its functional dependencies FD1 through FD4. (b) Decomposing into the 2NF relations LOTS1 and LOTS2. (c) Decomposing LOTS1 into the 3NF relations LOTS1A and LOTS1B. (d) Summary of the progressive normalization of LOTS.



| EmpID | Project ID | Performance Score | Increment - |
|-------|------------|-------------------|-------------|
| 1     | 50         | 85                | 100         |
| 1     | 51         | 90                | 150         |



## 3.4 Third Normal Form (1)

- Definition:

- **Transitive functional dependency:** a FD  $X \rightarrow Z$  that can be derived from two FDs  $X \rightarrow Y$  and  $Y \rightarrow Z$

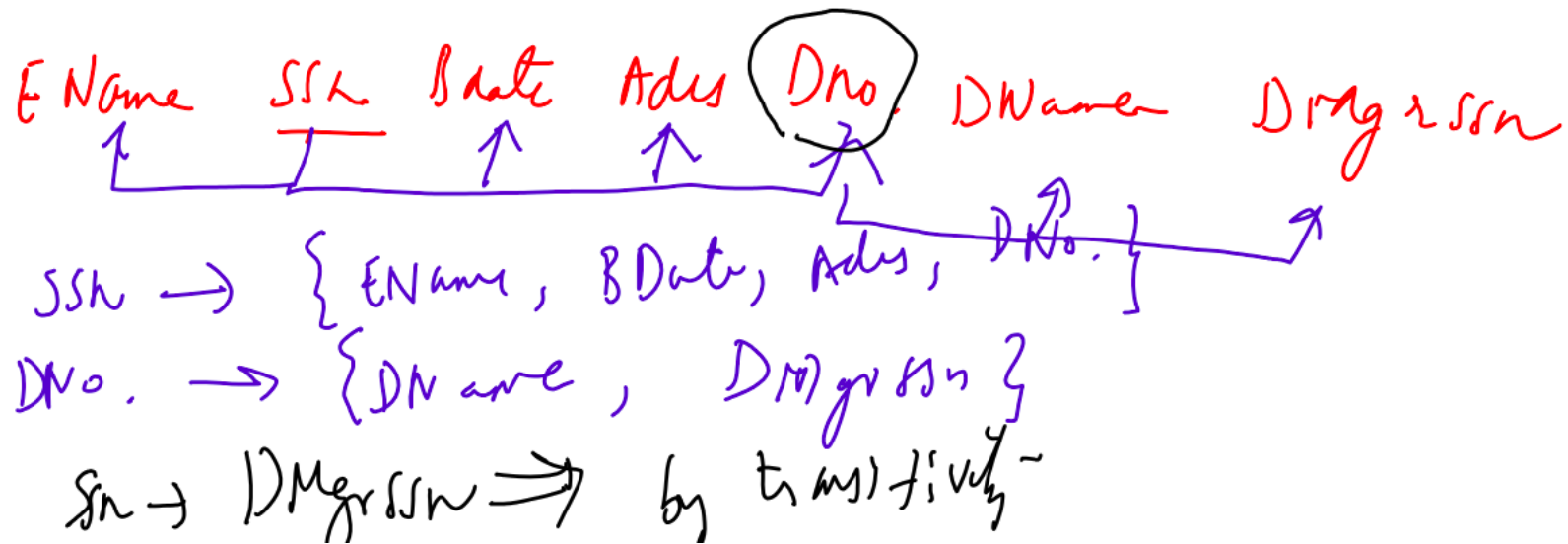
- Examples:

- $SSN \rightarrow DMGRSSN$  is a **transitive** FD
  - Since  $SSN \rightarrow DNUMBER$  and  $DNUMBER \rightarrow DMGRSSN$  hold
- $SSN \rightarrow ENAME$  is **non-transitive**
  - Since there is no set of attributes  $X$  where  $SSN \rightarrow X$  and  $X \rightarrow ENAME$

# Third Normal Form (2)

- A relation schema R is in **third normal form (3NF)** if it is in 2NF *and* no non-prime attribute A in R is transitively dependent on the primary key
- R can be decomposed into 3NF relations via the process of 3NF normalization
- NOTE:
  - In  $X \rightarrow Y$  and  $Y \rightarrow Z$ , with X as the primary key, we consider this a problem only if Y is not a candidate key.
  - When Y is a candidate key, there is no problem with the transitive dependency .
  - E.g., Consider EMP (SSN, Emp#, Salary ).
    - Here,  $SSN \rightarrow Emp\# \rightarrow Salary$  and Emp# is a candidate key.

An attribute that is part of any one candidate key is a Prime attribute



# Normal Forms Defined Informally

- 1<sup>st</sup> normal form
  - All attributes depend on **the key**
- 2<sup>nd</sup> normal form
  - All attributes depend on **the whole key**
- 3<sup>rd</sup> normal form
  - All attributes depend on **nothing but the key**

# General Normal Form Definitions 2NF and 3NF

- The above definitions consider the primary key only
- The following more general definitions take into account relations with multiple candidate keys
- A relation schema  $R$  is in **second normal form (2NF)** if every non-prime attribute  $A$  in  $R$  is fully functionally dependent on *every* key of  $R$

# General Normal Form Definitions

- Definition:  $\{FName, Eid\} \rightarrow \{ENAME\}$ 
  - **Superkey** of relation schema R - a set of attributes S of R that contains a key of R Trivial
  - A relation schema R is in **third normal form (3NF)** if whenever a FD  $X \rightarrow A$  holds in R, then either:
    - (a) X is a superkey of R, or
    - (b) A is a prime attribute of R
- NOTE: Boyce-Codd normal form disallows condition (b) above

# BCNF (Boyce-Codd Normal Form)

- A relation schema  $R$  is in **Boyce-Codd Normal Form (BCNF)** if whenever an **FD**  $X \rightarrow A$  holds in  $R$ , then  $X$  is a **superkey** of  $R$
- Each normal form is strictly stronger than the previous one
  - Every 2NF relation is in 1NF
  - Every 3NF relation is in 2NF
  - Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF
- The goal is to have each relation in BCNF (or 3NF)



# Figure 10.12 Boyce-Codd normal form

(a)

LOTS1A ✓

| <u>Property_id#</u> | County_name | Lot# | Area |
|---------------------|-------------|------|------|
|---------------------|-------------|------|------|

FD1

FD2

FD5

BCNF Normalization

LOTS1AX

| <u>Property_id#</u> | Area | Lot# |
|---------------------|------|------|
|---------------------|------|------|

LOTS1AY

| <u>Area</u> | County_name |
|-------------|-------------|
|-------------|-------------|

$\{ \text{County Name, Lot\#} \} \rightarrow \text{Area}$   
 $\{ \text{Area, Property ID} \} \rightarrow \text{Lot\#}$

$\text{Area} \rightarrow \text{County}$

Property ID → County Name  
 Property ID → Lot #  
 Property ID → Area

(b)

R

| <u>A</u> | <u>B</u> | C |
|----------|----------|---|
|----------|----------|---|

FD1

FD2

$\{ A, B \} \rightarrow C$   
 $C \rightarrow B$

**Figure 10.12**

Boyce-Codd normal form. (a) BCNF normalization of LOTS1A with the functional dependency FD2 being lost in the decomposition. (b) A schematic relation with FDs; it is in 3NF, but not in BCNF.

FD1: {Student, course}  $\rightarrow$  Instructor

FD2: Instructor  $\rightarrow$  course ✓

It's in 3NF, but not in BCNF

{Student, course}

$R_1 \cap R_2 = \text{Instructor}$

$R_1 - R_2 = \text{course}$

case 1

R1

|                |                   |
|----------------|-------------------|
| <u>Student</u> | <u>Instructor</u> |
|----------------|-------------------|

R2

|                |               |
|----------------|---------------|
| <u>Student</u> | <u>course</u> |
|----------------|---------------|

$R_1 \cap R_2 \rightarrow R_1 - R_2$

case 2

R1

|               |                   |
|---------------|-------------------|
| <u>course</u> | <u>Instructor</u> |
|---------------|-------------------|

R2

|               |                |
|---------------|----------------|
| <u>course</u> | <u>Student</u> |
|---------------|----------------|

Instructor  $\rightarrow$  course?

$R_1 \cap R_2 \rightarrow R_2 - R_1$

case 3

R1

|               |                   |
|---------------|-------------------|
| <u>course</u> | <u>Instructor</u> |
|---------------|-------------------|

R2

|                |                   |
|----------------|-------------------|
| <u>Student</u> | <u>Instructor</u> |
|----------------|-------------------|

Instructor  $\rightarrow$  student

# Figure 10.13 a relation TEACH that is in 3NF but not in BCNF

**TEACH**

| Student | Course            | Instructor |
|---------|-------------------|------------|
| Narayan | Database          | Mark       |
| Smith   | Database          | Navathe    |
| Smith   | Operating Systems | Ammar      |
| Smith   | Theory            | Schulman   |
| Wallace | Database          | Mark       |
| Wallace | Operating Systems | Ahamad     |
| Wong    | Database          | Omiecinski |
| Zelaya  | Database          | Navathe    |
| Narayan | Operating Systems | Ammar      |

**Figure 10.13**  
A relation TEACH that  
is in 3NF but not  
BCNF.

# Achieving the BCNF by Decomposition (1)

- Two FDs exist in the relation TEACH:
  - fd1: { student, course} -> instructor
  - fd2: instructor -> course
- {student, course} is a candidate key for this relation and that the dependencies shown follow the pattern in Figure 10.12 (b).
  - So this relation is in 3NF *but not in BCNF*
- A relation **NOT** in BCNF should be decomposed so as to meet this property, while possibly forgoing the preservation of all functional dependencies in the decomposed relations.
  - (See Algorithm 11.3)

# Achieving the BCNF by Decomposition (2)

- Three possible decompositions for relation TEACH
  - {student, instructor} and {student, course}
  - {course, instructor } and {course, student}
  - {instructor, course } and {instructor, student}
- All three decompositions will lose fd1.
  - We have to settle for sacrificing the functional dependency preservation. But we cannot sacrifice the non-additivity property after decomposition.
- Out of the above three, only the 3rd decomposition will not generate spurious tuples after join.(and hence has the non-additivity property).
- A test to determine whether a binary decomposition (decomposition into two relations) is non-additive (lossless) is discussed in section 11.1.4 under Property LJ1. Verify that the third decomposition above meets the property.

# Chapter Outline

- Informal Design Guidelines for Relational Databases
- Functional Dependencies (FDs)
  - Definition, Inference Rules, Equivalence of Sets of FDs, Minimal Sets of FDs
- Normal Forms Based on Primary Keys
- General Normal Form Definitions (For Multiple Keys)
- BCNF (Boyce-Codd Normal Form)

*Fundamentals of*  
**Database  
Systems**

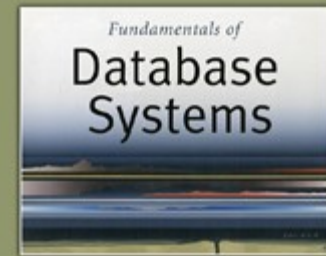


5<sup>th</sup> Edition

Elmasri / Navathe

# Chapter 11

## Relational Database Design Algorithms and Further Dependencies



5th Edition

Elmasri / Navathe



# Chapter Outline

- Properties of Relational Decompositions
- Multivalued Dependencies and Fourth Normal Form
- Join Dependencies and Fifth Normal Form

# Properties of Relational Decompositions (1)

- **Relation Decomposition and Insufficiency of Normal Forms:**
  - **Universal Relation Schema:**
    - A relation schema  $R = \{A_1, A_2, \dots, A_n\}$  that includes all the attributes of the database.
  - **Universal relation assumption:**
    - Every attribute name is unique.

# Properties of Relational Decompositions (2)

- **Relation Decomposition and Insufficiency of Normal Forms (cont.):**
  - **Decomposition:**
    - The process of decomposing the universal relation schema  $R$  into a set of relation schemas  $D = \{R_1, R_2, \dots, R_m\}$  that will become the relational database schema by using the functional dependencies.
  - **Attribute preservation condition:**
    - Each attribute in  $R$  will appear in at least one relation schema  $R_i$  in the decomposition so that no attributes are “lost”.

# Properties of Relational Decompositions (2)

- Another goal of decomposition is to have each individual relation  $R_i$  in the decomposition  $D$  be in BCNF or 3NF.
- Additional properties of decomposition are needed to prevent from generating spurious tuples

# Properties of Relational Decompositions (3)

## ■ Dependency Preservation Property of a Decomposition:

- Definition: Given a set of dependencies  $F$  on  $R$ , the **projection** of  $F$  on  $R_i$ , denoted by  $p_{R_i}(F)$  where  $R_i$  is a subset of  $R$ , is the set of dependencies  $X \twoheadrightarrow Y$  in  $F^+$  such that the attributes in  $X \cup Y$  are all contained in  $R_i$ .
- Hence, the projection of  $F$  on each relation schema  $R_i$  in the decomposition  $D$  is the set of functional dependencies in  $F^+$ , the closure of  $F$ , such that all their left- and right-hand-side attributes are in  $R_i$ .

# Properties of Relational Decompositions (4)

- **Dependency Preservation Property of a Decomposition (cont.):**
  - **Dependency Preservation Property:**
    - A decomposition  $D = \{R_1, R_2, \dots, R_m\}$  of  $R$  is **dependency-preserving** with respect to  $F$  if the union of the projections of  $F$  on each  $R_i$  in  $D$  is equivalent to  $F$ ; that is
$$((\pi_{R_1}(F)) \cup \dots \cup (\pi_{R_m}(F)))^+ = F^+$$
    - (See examples in Fig 10.12a and Fig 10.11)
- **Claim 1:**
  - It is always possible to find a dependency-preserving decomposition  $D$  with respect to  $F$  such that each relation  $R_i$  in  $D$  is in 3nf.

# Properties of Relational Decompositions (5)

## ■ Lossless (Non-additive) Join Property of a Decomposition:

- Definition: Lossless join property: a decomposition  $D = \{R_1, R_2, \dots, R_m\}$  of  $R$  has the **lossless (nonadditive) join property** with respect to the set of dependencies  $F$  on  $R$  if, for every relation state  $r$  of  $R$  that satisfies  $F$ , the following holds, where  $*$  is the natural join of all the relations in  $D$ :

$$* (\pi_{R_1}(r), \dots, \pi_{R_m}(r)) = r$$

- Note: The word loss in lossless refers to loss of information, not to loss of tuples. In fact, for “loss of information” a better term is “**addition of spurious information**”

# Properties of Relational Decompositions (9)

- **Testing Binary Decompositions for Lossless Join Property**
  - **Binary Decomposition:** Decomposition of a relation  $R$  into two relations.
  - **PROPERTY LJ1 (lossless join test for binary decompositions):** A decomposition  $D = \{R_1, R_2\}$  of  $R$  has the lossless join property with respect to a set of functional dependencies  $F$  on  $R$  *if and only if* either
    - The f.d.  $((R_1 \cap R_2) \rightarrow (R_1 - R_2))$  is in  $F^+$ , or
    - The f.d.  $((R_1 \cap R_2) \rightarrow (R_2 - R_1))$  is in  $F^+$ .





# Multivalued Dependencies and Fourth Normal Form (2)

## Definition:

- A **multivalued dependency (MVD)**  $X \twoheadrightarrow Y$  specified on relation schema  $R$ , where  $X$  and  $Y$  are both subsets of  $R$ , specifies the following constraint on any relation state  $r$  of  $R$ : If two tuples  $t_1$  and  $t_2$  exist in  $r$  such that  $t_1[X] = t_2[X]$ , then two tuples  $t_3$  and  $t_4$  should also exist in  $r$  with the following properties, where we use  $Z$  to denote  $(R - (X \cup Y))$ :
  - $t_3[X] = t_4[X] = t_1[X] = t_2[X]$ .
  - $t_3[Y] = t_1[Y]$  and  $t_4[Y] = t_2[Y]$ .
  - $t_3[Z] = t_2[Z]$  and  $t_4[Z] = t_1[Z]$ .
- An MVD  $X \twoheadrightarrow Y$  in  $R$  is called a **trivial MVD** if (a)  $Y$  is a subset of  $X$ , or (b)  $X \cup Y = R$ .

# Multivalued Dependencies and Fourth Normal Form (3)

- **Inference Rules for Functional and Multivalued Dependencies:**
  - IR1 (reflexive rule for FDs): If  $X \supseteq Y$ , then  $X \rightarrow Y$ .
  - IR2 (augmentation rule for FDs):  $\{X \rightarrow Y\} \models XZ \rightarrow YZ$ .
  - IR3 (transitive rule for FDs):  $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow Z$ .
  - IR4 (complementation rule for MVDs):  $\{X \twoheadrightarrow Y\} \models X \twoheadrightarrow (R - (X \cup Y))$ .
  - IR5 (augmentation rule for MVDs): If  $X \twoheadrightarrow Y$  and  $W \supseteq Z$  then  $WX \twoheadrightarrow YZ$ .
  - IR6 (transitive rule for MVDs):  $\{X \twoheadrightarrow Y, Y \twoheadrightarrow Z\} \models X \twoheadrightarrow (Z \sqcap Y)$ .
  - IR7 (replication rule for FD to MVD):  $\{X \rightarrow Y\} \models X \twoheadrightarrow Y$ .
  - IR8 (coalescence rule for FDs and MVDs): If  $X \twoheadrightarrow Y$  and there exists  $W$  with the properties that
    - (a)  $W \cap Y$  is empty, (b)  $W \rightarrow Z$ , and (c)  $Y \supseteq Z$ , then  $X \rightarrow Z$ .

# Multivalued Dependencies and Fourth Normal Form (4)

## Definition:

- A relation schema  $R$  is in **4NF** with respect to a set of dependencies  $F$  (that includes functional dependencies and multivalued dependencies) if, for every *nontrivial* multivalued dependency  $X \twoheadrightarrow Y$  in  $F^+$ ,  $X$  is a superkey for  $R$ .
- Note:  $F^+$  is the (complete) set of all dependencies (functional or multivalued) that will hold in every relation state  $r$  of  $R$  that satisfies  $F$ . It is also called the **closure** of  $F$ .

# Multivalued Dependencies and Fourth Normal Form (5)

Decomposing a relation state of EMP that is not in 4NF:

- (a) EMP relation with additional tuples.
- (b) Two corresponding 4NF relations EMP\_PROJECTS and EMP\_DEPENDENTS.

(a) **EMP**

| <u>ENAME</u> | PNAME | <u>DNAME</u> |
|--------------|-------|--------------|
| Smith        | X     | John         |
| Smith        | Y     | Anna         |
| Smith        | X     | Anna         |
| Smith        | Y     | John         |
| Brown        | W     | Jim          |
| Brown        | X     | Jim          |
| Brown        | Y     | Jim          |
| Brown        | Z     | Jim          |
| Brown        | W     | Joan         |
| Brown        | X     | Joan         |
| Brown        | Y     | Joan         |
| Brown        | Z     | Joan         |
| Brown        | W     | Bob          |
| Brown        | X     | Bob          |
| Brown        | Y     | Bob          |
| Brown        | Z     | Bob          |

(b) **EMP\_PROJECTS**

| <u>ENAME</u> | <u>PNAME</u> |
|--------------|--------------|
| Smith        | X            |
| Smith        | Y            |
| Brown        | W            |
| Brown        | X            |
| Brown        | Y            |
| Brown        | Z            |

**EMP\_DEPENDENTS**

| <u>ENAME</u> | <u>DNAME</u> |
|--------------|--------------|
| Smith        | Anna         |
| Smith        | John         |
| Brown        | Jim          |
| Brown        | Joan         |
| Brown        | Bob          |

## 4. Join Dependencies and Fifth Normal Form (1)

### Definition:

- A **join dependency (JD)**, denoted by  $JD(R_1, R_2, \dots, R_n)$ , specified on relation schema  $R$ , specifies a constraint on the states  $r$  of  $R$ .
- The constraint states that every legal state  $r$  of  $R$  should have a non-additive join decomposition into  $R_1, R_2, \dots, R_n$ ; that is, for every such  $r$  we have
- $$* (\pi_{R_1}(r), \pi_{R_2}(r), \dots, \pi_{R_n}(r)) = r$$

**Note:** an MVD is a special case of a JD where  $n = 2$ .

- A join dependency  $JD(R_1, R_2, \dots, R_n)$ , specified on relation schema  $R$ , is a **trivial JD** if one of the relation schemas  $R_i$  in  $JD(R_1, R_2, \dots, R_n)$  is equal to  $R$ .

## Join Dependencies and Fifth Normal Form (2)

### Definition:

- A relation schema  $R$  is in **fifth normal form (5NF)** (or **Project-Join Normal Form (PJNF)**) with respect to a set  $F$  of functional, multivalued, and join dependencies if,
  - for every nontrivial join dependency  $JD(R_1, R_2, \dots, R_n)$  in  $F^+$  (that is, implied by  $F$ ),
    - every  $R_i$  is a superkey of  $R$ .

# Relation SUPPLY with Join Dependency and conversion to Fifth Normal Form

**Figure 11.4**

Fourth and fifth normal forms.

- (a) The EMP relation with two MVDs:  $\text{Ename} \twoheadrightarrow \text{Pname}$  and  $\text{Ename} \twoheadrightarrow \text{Dname}$ .
- (b) Decomposing the EMP relation into two 4NF relations EMP\_PROJECTS and EMP\_DEPENDENTS.
- (c) The relation SUPPLY with no MVDs is in 4NF but not in 5NF if it has the JD( $R_1, R_2, R_3$ ).
- (d) Decomposing the relation SUPPLY into the 5NF relations  $R_1, R_2, R_3$ .

**(c) SUPPLY**

| <u>Sname</u> | <u>Part_name</u> | <u>Proj_name</u> |
|--------------|------------------|------------------|
| Smith        | Bolt             | ProjX            |
| Smith        | Nut              | ProjY            |
| Adamsky      | Bolt             | ProjY            |
| Walton       | Nut              | ProjZ            |
| Adamsky      | Nail             | ProjX            |
| Adamsky      | Bolt             | ProjX            |
| Smith        | Bolt             | ProjY            |

**(d)  $R_1$**

| <u>Sname</u> | <u>Part_name</u> |
|--------------|------------------|
| Smith        | Bolt             |
| Smith        | Nut              |
| Adamsky      | Bolt             |
| Walton       | Nut              |
| Adamsky      | Nail             |

**$R_2$**

| <u>Sname</u> | <u>Proj_name</u> |
|--------------|------------------|
| Smith        | ProjX            |
| Smith        | ProjY            |
| Adamsky      | ProjY            |
| Walton       | ProjZ            |
| Adamsky      | ProjX            |

**$R_3$**

| <u>Part_name</u> | <u>Proj_name</u> |
|------------------|------------------|
| Bolt             | ProjX            |
| Nut              | ProjY            |
| Bolt             | ProjY            |
| Nut              | ProjZ            |
| Nail             | ProjX            |



# Recap

- Designing a Set of Relations
- Properties of Relational Decompositions
- Algorithms for Relational Database Schema
- Multivalued Dependencies and Fourth Normal Form
- Join Dependencies and Fifth Normal Form
- Inclusion Dependencies
- Other Dependencies and Normal Forms